Software Engineering Case Study: IVF Fertility Hospital Management System

Document type: System modelling and architectural case study
Domain: Hospital Management System (IVF Fertility)
Modelling standard: UML 2.x, relational data model

---

1 System Overview

1.0.1 1.1 Problem Definition

An IVF fertility clinic must coordinate patient registration, clinical consultations, laboratory procedures (embryo culture, grading, cryopreservation), donor bank inventory, equipment usage and maintenance, test reporting, and billing. Data is held in paper records, spreadsheets, and isolated systems. This leads to inconsistent patient records, untraceable embryo handling, billing errors, and no single source of truth for cycle status or equipment involvement in procedures.

The system must replace fragmented processes with a centralized Hospital Management System (HMS) that enforces traceability, supports audit trails, and maintains referential integrity across patient, laboratory, and administrative data.

1.0.2 1.2 System Purpose

The IVF Fertility HMS shall:

- Register and maintain patient, doctor, and donor entities with normalized contact and demographic data.
- Link patients to rooms, medical reports, test reports, and embryos so that every clinical event is attributable to a patient and (where applicable) to equipment and staff.
- Manage embryo records by status, stage, cycle, and incubator (equipment), supporting chain-of-custody and viability tracking.
- Track which equipment is used in which test reports (Used_In) and which doctors maintain which equipment (Maintained).
- Support donation bank inventory (Donation_Bank, Donates) and donor payments (Donor_Payment, Service).
- Generate and associate patient bills with services (Patient_Bill, Service) and support billing queries.

The system is a data-centric control and information management system: it records and links entities and events for accountability and reporting.

1.1 2. Functional Requirements

1. The system shall allow registration of a patient with P_ID, Name, DOB, Sex, Age, Date_Admitted, RoomNr, and optional R_ID (Medical_Report).
2. The system shall allow zero or more phone numbers per patient (Patient_PhNr) and per doctor (Doctor_PhNr) and per donor (Donor_PhNr).
3. The system shall assign each patient to exactly one room (RoomNr) and support room attributes: Availability, RoomType, Cost, Capacity.
4. The system shall allow creation of a medical report with R_ID, Appointment date, and Description, and optional link from Patient to Medical_Report (R_ID).
5. The system shall allow creation of a test report with Report_ID, T_Result, Appointment date, Description, and P_ID (patient).
6. The system shall allow association of one or more equipment items with a test report (Used_In: Eq_ID, Report_ID).
7. The system shall store embryo records with E_ID, Status, Stage, CycleID, Eq_ID (equipment), and P_ID (patient).
8. The system shall allow doctors to be associated with specializations (Doctor_Specialization: S_ID, Department, DR_ID).
9. The system shall allow doctors to be associated with equipment they maintain (Maintained: DR_ID, Eq_ID).
10. The system shall maintain equipment master data (Eq_ID, Name, Quantity, Condition).
11. The system shall maintain donation banks (DB_ID, Issued, Ordered, InStock, Storage_Capacity) and donor–bank contributions (Donates: D_ID, DB_ID).
12. The system shall allow donor payment records (Donor_Payment: Payment_ID, Payment_Method, Service_ID, D_ID).
13. The system shall maintain a service catalogue (Service_ID, Description, Cost) and patient bills (Patient_Bill: PB_ID, P_ID, Service_ID).
14. The system shall enforce foreign key constraints so that RoomNr in Patient references Room, R_ID in Patient references Medical_Report, and P_ID in Test_Report, Embryo, and Patient_Bill references Patient.
15. The system shall support queries that aggregate bills by patient (sum of service costs per P_ID) and that list test reports by patient and by result (e.g. T_Result).
16. The system shall support queries that determine which doctors maintain equipment used in test reports with a given result (e.g. Positive) for traceability.

1.2 3. System Scenarios

1.2.1 3.1 Scenario 1: Appointment Booking and Consultation Workflow

Initial assumptions: Patient exists (P_ID). Room exists and is available. Medical report does not yet exist for this consultation.

Normal flow:

1. Administrator or clinician creates a Medical_Report with R_ID, Appointment (date), Description.
2. Patient record is updated with R_ID pointing to this Medical_Report (if one patient is linked per report).
3. Patient is already assigned to a room (RoomNr). No room change in this scenario.
4. System state: Patient has valid RoomNr and R_ID; Medical_Report exists with Appointment and Description.

What can go wrong:

- Room not available: Room.Availability = 'No' for chosen room; assignment may be rejected or another room selected.
- Duplicate R_ID: Primary key violation on Medical_Report; creation fails.
- Invalid RoomNr: Foreign key violation if Patient.RoomNr does not exist in Room.

System state after completion: Patient.R_ID set to new Medical_Report.R_ID; one new row in Medical_Report.

1.2.2 3.2 Scenario 2: Lab Test Upload and Equipment Association Workflow

Initial assumptions: Patient exists. At least one Equipment record exists. Test report does not yet exist.

Normal flow:

1. Lab technician creates a Test_Report: Report_ID, T_Result (e.g. Positive, Negative, Pending), Appointment date, Description, P_ID.
2. Technician records which equipment was used for the test by inserting into Used_In (Eq_ID, Report_ID) for each piece of equipment.
3. System enforces foreign keys: Eq_ID in Equipment, Report_ID in Test_Report.
4. Queries can retrieve tests by patient, by result, and by equipment used.

What can go wrong:

- Invalid P_ID: Insert into Test_Report fails.
- Invalid Eq_ID or Report_ID in Used_In: Insert fails.
- Report_ID not yet committed: If Used_In is inserted in same transaction after Test_Report, ordering is correct; otherwise failure.
- Equipment omitted: Used_In incomplete; audit trail incomplete.

System state after completion: One Test_Report row; one or more Used_In rows linking that report to equipment.

1.2.3 3.3 Scenario 3: Billing Generation Workflow

Initial assumptions: Patient exists. Service(s) exist. Patient_Bill rows may already exist for the patient.

Normal flow:

1. Administrator selects a patient (P_ID) and a service (Service_ID) and creates a Patient_Bill row: PB_ID, P_ID, Service_ID.
2. System enforces foreign keys to Patient and Service.
3. Total amount for the patient is computed by summing Service.Cost for all Patient_Bill rows with that P_ID.
4. No duplicate PB_ID (primary key).

What can go wrong:

- Invalid P_ID or Service_ID: Insert fails.
- Duplicate PB_ID: Primary key violation.
- Wrong Service_ID: Patient charged for wrong service until corrected; reconciliation required.

- No referential integrity to “cycle” or “appointment”: Bill not automatically linked to a specific cycle or visit; business logic must enforce that in application layer if required.

System state after completion: New Patient_Bill row; patient’s total bill increases by the selected Service.Cost.

1.3 4. Use Case Diagram

Actors: Patient, Doctor, Hospital Administrator, Lab Technician. Key use cases: Register Patient, Assign Room, Create Medical Report, Create Test Report, Associate Equipment to Test, Record Embryo, Manage Donation Bank, Generate Patient Bill, Record Donor Payment, Maintain Equipment, View Patient Reports, View Embryo Status. Include/extend as below.

1.3.0.1 PlantUML

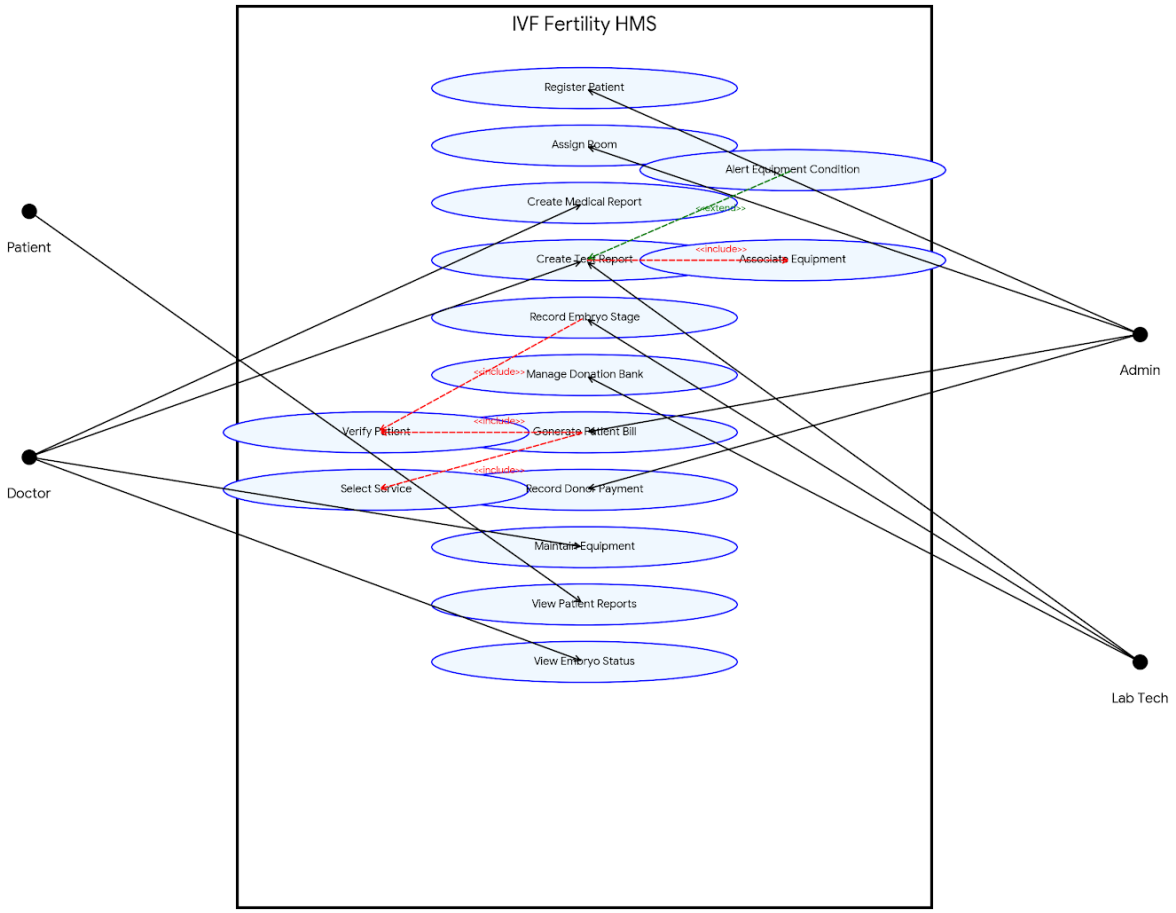
```
\NormalTok{@startuml IVF\_HMS\_UseCase}
\NormalTok{left to right direction}
\NormalTok{skinparam packageStyle rectangle}
\NormalTok{actor "Patient" as P}
\NormalTok{actor "Doctor" as D}
\NormalTok{actor "Hospital Administrator" as A}
\NormalTok{actor "Lab Technician" as L}

\NormalTok{rectangle "IVF Fertility HMS" \{}
\NormalTok{ usecase "Register Patient" as UC1}
\NormalTok{ usecase "Assign Room" as UC2}
\NormalTok{ usecase "Create Medical Report" as UC3}
\NormalTok{ usecase "Create Test Report" as UC4}
\NormalTok{ usecase "Associate Equipment to Test" as UC5}
\NormalTok{ usecase "Record Embryo Stage" as UC6}
\NormalTok{ usecase "Manage Donation Bank" as UC7}
\NormalTok{ usecase "Generate Patient Bill" as UC8}
\NormalTok{ usecase "Record Donor Payment" as UC9}
\NormalTok{ usecase "Maintain Equipment" as UC10}
\NormalTok{ usecase "View Patient Reports" as UC11}
\NormalTok{ usecase "View Embryo Status" as UC12}
\NormalTok{ usecase "Verify Patient" as UC13}
\NormalTok{ usecase "Select Service" as UC14}
\NormalTok{ usecase "Alert Equipment Condition" as UC15}

\NormalTok{ UC8 ..\textgreater{} UC13 : \textless{}\textless{}include\textgreater{}\textgreater{}}
\NormalTok{ UC8 ..\textgreater{} UC14 : \textless{}\textless{}include\textgreater{}\textgreater{}}
\NormalTok{ UC4 ..\textgreater{} UC5 : \textless{}\textless{}include\textgreater{}\textgreater{}}
\NormalTok{ UC6 ..\textgreater{} UC13 : \textless{}\textless{}include\textgreater{}\textgreater{}}
\NormalTok{ UC4 ..\textgreater{} UC15 : \textless{}\textless{}extend\textgreater{}\textgreater{}}
\NormalTok{\}}

\NormalTok{P {-}{-}\textgreater{} UC11}
\NormalTok{D {-}{-}\textgreater{} UC3}
\NormalTok{D {-}{-}\textgreater{} UC4}
\NormalTok{D {-}{-}\textgreater{} UC12}
\NormalTok{D {-}{-}\textgreater{} UC10}
\NormalTok{A {-}{-}\textgreater{} UC1}
\NormalTok{A {-}{-}\textgreater{} UC2}
\NormalTok{A {-}{-}\textgreater{} UC8}
\NormalTok{L {-}{-}\textgreater{} UC4}
```

```
\NormalTok{L {-}{-}\textgreater{} UC6}
\NormalTok{L {-}{-}\textgreater{} UC7}
\NormalTok{A {-}{-}\textgreater{} UC9}
\NormalTok{@enduml}
```

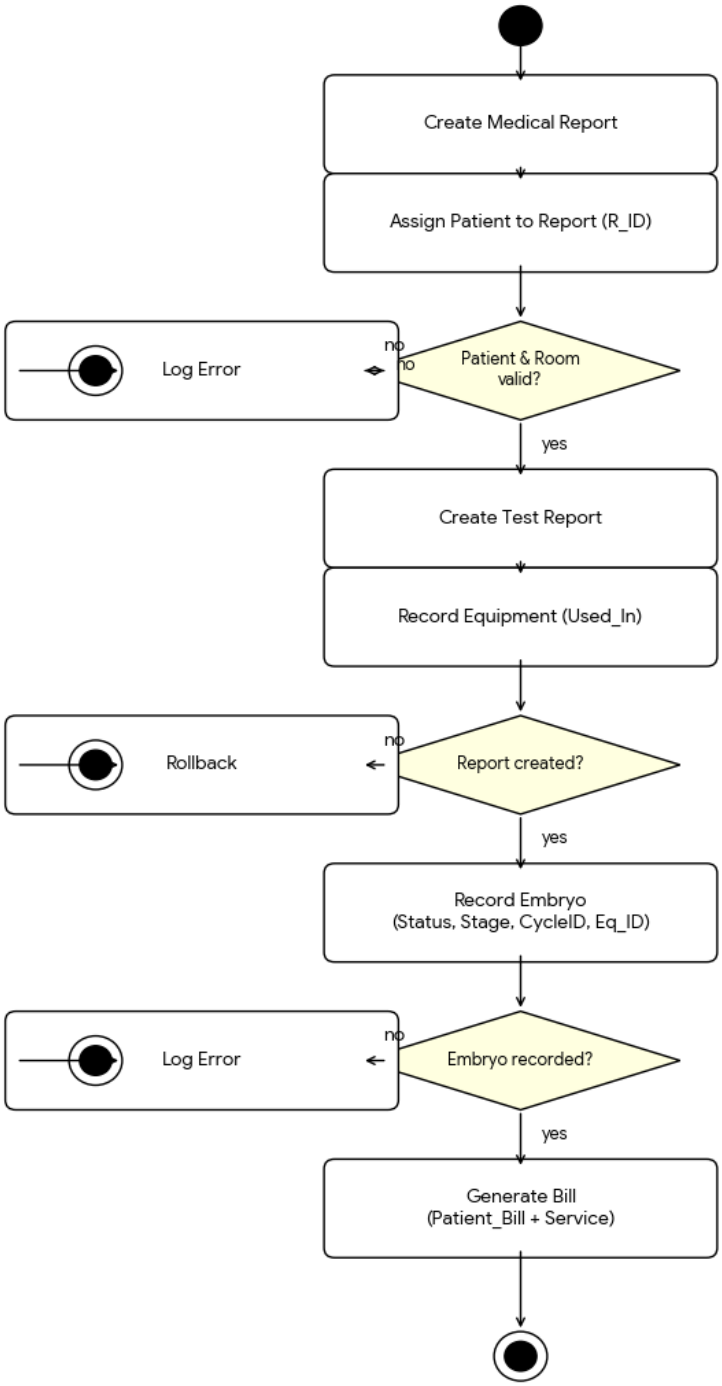


1.4 5. Activity Diagram

IVF treatment flow: create medical report, assign patient, create test report and record equipment (Used_In), record embryo, generate bill. Decision nodes: patient/room valid, report created, embryo recorded; error paths: log error or rollback.

1.4.0.1 PlantUML

```
\NormalTok{@startuml IVF\_Treatment\_Lifecycle}
\NormalTok{start}
\NormalTok{:Create Medical Report;}
\NormalTok{:Assign Patient to Report (R\_ID);}
\NormalTok{if (Patient \& Room valid?) then (yes)}
\NormalTok{ :Create Test Report;}
\NormalTok{ :Record Equipment (Used\_In);}
\NormalTok{ if (Report created?) then (yes)}
\NormalTok{    :Record Embryo (Status, Stage, CycleID, Eq\_ID);}
\NormalTok{    if (Embryo recorded?) then (yes)}
\NormalTok{      :Generate Bill (Patient\_Bill + Service);}
\NormalTok{      stop}
\NormalTok{    else (no)}
\NormalTok{      :Log Error;}
\NormalTok{      stop}
\NormalTok{    endif}
\NormalTok{ else (no)}
\NormalTok{    :Rollback;}
\NormalTok{    stop}
\NormalTok{ endif}
\NormalTok{else (no)}
\NormalTok{ :Log Error;}
\NormalTok{ stop}
\NormalTok{endif}
\NormalTok{@enduml}
```


Create Medical Report
Assign Patient to Report (R_ID)
Patient & Room valid?
no
Log Error
yes
Create Test Report
Record Equipment (Used_In)
Report created?
no
Rollback
yes
Record Embryo
(Status, Stage, CycleID, Eq_ID)
Embryo recorded?
no
Log Error
yes
Generate Bill
(Patient_Bill + Service)

1.5 6. Class Diagram

The class diagram is derived strictly from the SQL schema. Tables map to classes; primary and foreign keys drive associations and multiplicities. Multi-valued attributes (phone numbers) and many-to-many relationships (Donates, Maintained, Used_In) appear as separate classes or associations.

1.5.0.1 PlantUML

```
\NormalTok{@startuml IVF\_HMS\_ClassDiagram}
\NormalTok{class Patient \{}
\NormalTok{ {-} P\_ID : INT}
\NormalTok{  {-} Name : VARCHAR(50)}
\NormalTok{  {-} DOB : DATE}
\NormalTok{ {-} Sex : CHAR(1)}
```

```
\NormalTok{ {-} Age : INT}
\NormalTok{ {-} Date\_Admitted : DATE}
\NormalTok{ {-} Date\_Discharged : DATE}
\NormalTok{ {-} RoomNr : INT}
\NormalTok{ {-} R\_ID : VARCHAR(20)}
\NormalTok{ + register()}
\NormalTok{ + getBills()}
\NormalTok{ + getTestReports()}
\NormalTok{ + getEmbryos()}
\NormalTok{\}}

\NormalTok{class Room \{}
\NormalTok{ {-} RoomNr : INT}
\NormalTok{ {-} Availability : VARCHAR(10)}
\NormalTok{ {-} RoomType : VARCHAR(20)}
\NormalTok{ {-} Cost : INT}
\NormalTok{ {-} Capacity : INT}
\NormalTok{ + isAvailable()}
\NormalTok{\}}

\NormalTok{class Medical\_Report \{}
\NormalTok{ {-} R\_ID : VARCHAR(20)}
\NormalTok{ {-} Appointment : DATE}
\NormalTok{ {-} Description : VARCHAR(100)}
\NormalTok{ + getPatient()}
\NormalTok{\}}

\NormalTok{class Test\_Report \{}
\NormalTok{ {-} Report\_ID : VARCHAR(20)}
\NormalTok{ {-} T\_Result : VARCHAR(20)}
\NormalTok{ {-} Appointment : DATE}
\NormalTok{ {-} Description : VARCHAR(100)}
\NormalTok{ {-} P\_ID : INT}
\NormalTok{ + getEquipmentUsed()}
\NormalTok{ + getPatient()}
\NormalTok{\}}

\NormalTok{class Embryo \{}
\NormalTok{ {-} E\_ID : VARCHAR(10)}
\NormalTok{ {-} Status : VARCHAR(50)}
\NormalTok{ {-} Stage : VARCHAR(50)}
\NormalTok{ {-} CycleID : VARCHAR(10)}
\NormalTok{ {-} Eq\_ID : VARCHAR(10)}
\NormalTok{ {-} P\_ID : INT}
\NormalTok{ + recordDevelopment()}
\NormalTok{ + getPatient()}
\NormalTok{ + getEquipment()}
\NormalTok{\}}

\NormalTok{class Equipment \{}
\NormalTok{ {-} Eq\_ID : VARCHAR(10)}
\NormalTok{ {-} Name : VARCHAR(50)}
\NormalTok{ {-} Quantity : INT}
\NormalTok{ {-} Condition : VARCHAR(20)}
\NormalTok{ + getTestsUsedIn()}
\NormalTok{ + getMaintainers()}
\NormalTok{\}}
```

```
\NormalTok{class Service \{}
\NormalTok{ {-} Service\_ID : VARCHAR(10)}
\NormalTok{ {-} Description : VARCHAR(25)}
\NormalTok{ {-} Cost : INT}
\NormalTok{ + getBills()}
\NormalTok{\}}

\NormalTok{class Patient\_Bill \{}
\NormalTok{ {-} PB\_ID : VARCHAR(10)}
\NormalTok{ {-} P\_ID : INT}
\NormalTok{ {-} Service\_ID : VARCHAR(10)}
\NormalTok{ + calculateTotal()}
\NormalTok{\}}

\NormalTok{class Doctor \{}
\NormalTok{ {-} DR\_ID : INT}
\NormalTok{ {-} Name : VARCHAR(25)}
\NormalTok{ {-} DOB : DATE}
\NormalTok{ {-} Sex : CHAR(1)}
\NormalTok{ {-} Age : INT}
\NormalTok{ {-} City : VARCHAR(50)}
\NormalTok{ {-} County : VARCHAR(50)}
\NormalTok{ {-} ZipCode : VARCHAR(10)}
\NormalTok{ {-} Salary : INT}
\NormalTok{ + getSpecializations()}
\NormalTok{ + getMaintainedEquipment()}
\NormalTok{\}}

\NormalTok{class Doctor\_Specialization \{}
\NormalTok{ {-} S\_ID : INT}
\NormalTok{ {-} Department : VARCHAR(50)}
\NormalTok{ {-} DR\_ID : INT}
\NormalTok{\}}

\NormalTok{class Donation\_Bank \{}
\NormalTok{ {-} DB\_ID : VARCHAR(10)}
\NormalTok{ {-} Issued : DATE}
\NormalTok{ {-} Ordered : DATE}
\NormalTok{ {-} InStock : INT}
\NormalTok{ {-} Storage\_Capacity : INT}
\NormalTok{ + getDonors()}
\NormalTok{ + updateStock()}
\NormalTok{\}}

\NormalTok{class Donor \{}
\NormalTok{ {-} D\_ID : INT}
\NormalTok{ {-} Name : VARCHAR(50)}
\NormalTok{ {-} DOB : DATE}
\NormalTok{ {-} Sex : CHAR(1)}
\NormalTok{ {-} Age : INT}
\NormalTok{ {-} RoomNr : INT}
\NormalTok{ + getBanks()}
\NormalTok{ + getPayments()}
\NormalTok{\}}

\NormalTok{class Donor\_Payment \{}
\NormalTok{ {-} Payment\_ID : VARCHAR(10)}
\NormalTok{ {-} Payment\_Method : VARCHAR(50)}
```

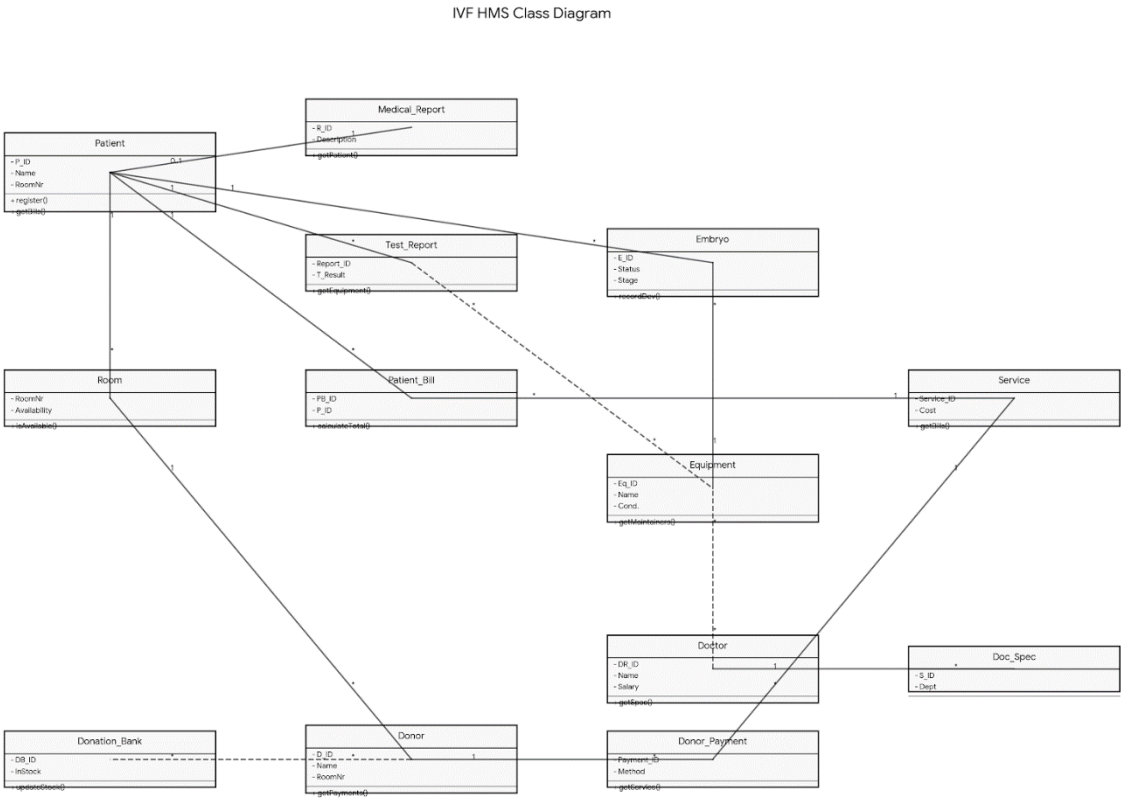
```
\NormalTok{  {-}  Service\_ID  :  VARCHAR(10)}
\NormalTok{  {-}  D\_ID  :  INT}
\NormalTok{  +  getDonor()}
\NormalTok{  +  getService()}
\NormalTok{\}}

\NormalTok{class Used\_In \{}
\NormalTok{  {-}  Eq\_ID  :  VARCHAR(10)}
\NormalTok{  {-}  Report\_ID  :  VARCHAR(20)}
\NormalTok{\}}

\NormalTok{class  Maintained  \{}
\NormalTok{  {-}  DR\_ID  :  INT}
\NormalTok{  {-}  Eq\_ID  :  VARCHAR(10)}
\NormalTok{\}}

\NormalTok{class  Donates  \{}
\NormalTok{  {-}  D\_ID  :  INT}
\NormalTok{  {-}  DB\_ID  :  VARCHAR(10)}
\NormalTok{\}}

\NormalTok{Patient "1" {-}{-}\textgreater{} "*" Room : RoomNr}
\NormalTok{Patient "0..1" {-}{-}\textgreater{} "1" Medical\_Report : R\_ID}
\NormalTok{Patient "1" {-}{-}\textgreater{} "*" Test\_Report : P\_ID}
\NormalTok{Patient "1" {-}{-}\textgreater{} "*" Embryo : P\_ID}
\NormalTok{Patient "1" {-}{-}\textgreater{} "*" Patient\_Bill : P\_ID}
\NormalTok{Test\_Report "*" {-}{-}\textgreater{} "*" Equipment : Used\_In}
\NormalTok{Embryo "*" {-}{-}\textgreater{} "1" Equipment : Eq\_ID}
\NormalTok{Patient\_Bill "*" {-}{-}\textgreater{} "1" Service : Service\_ID}
\NormalTok{Doctor "1" {-}{-}\textgreater{} "*" Doctor\_Specialization : DR\_ID}
\NormalTok{Doctor "*" {-}{-}\textgreater{} "*" Equipment : Maintained}
\NormalTok{Donor "*" {-}{-}\textgreater{} "*" Donation\_Bank : Donates}
\NormalTok{Donor "1" {-}{-}\textgreater{} "*" Donor\_Payment : D\_ID}
\NormalTok{Donor\_Payment "*" {-}{-}\textgreater{} "1" Service : Service\_ID}
\NormalTok{Donor "*" {-}{-}\textgreater{} "1" Room : RoomNr}
\NormalTok{@enduml}
```



1.6 7. Sequence Diagrams

1.6.1 7.1 Book Appointment

Participants: User (Administrator/Doctor), HMS UI, AppointmentController (or equivalent), Patient, Medical_Report, Database.

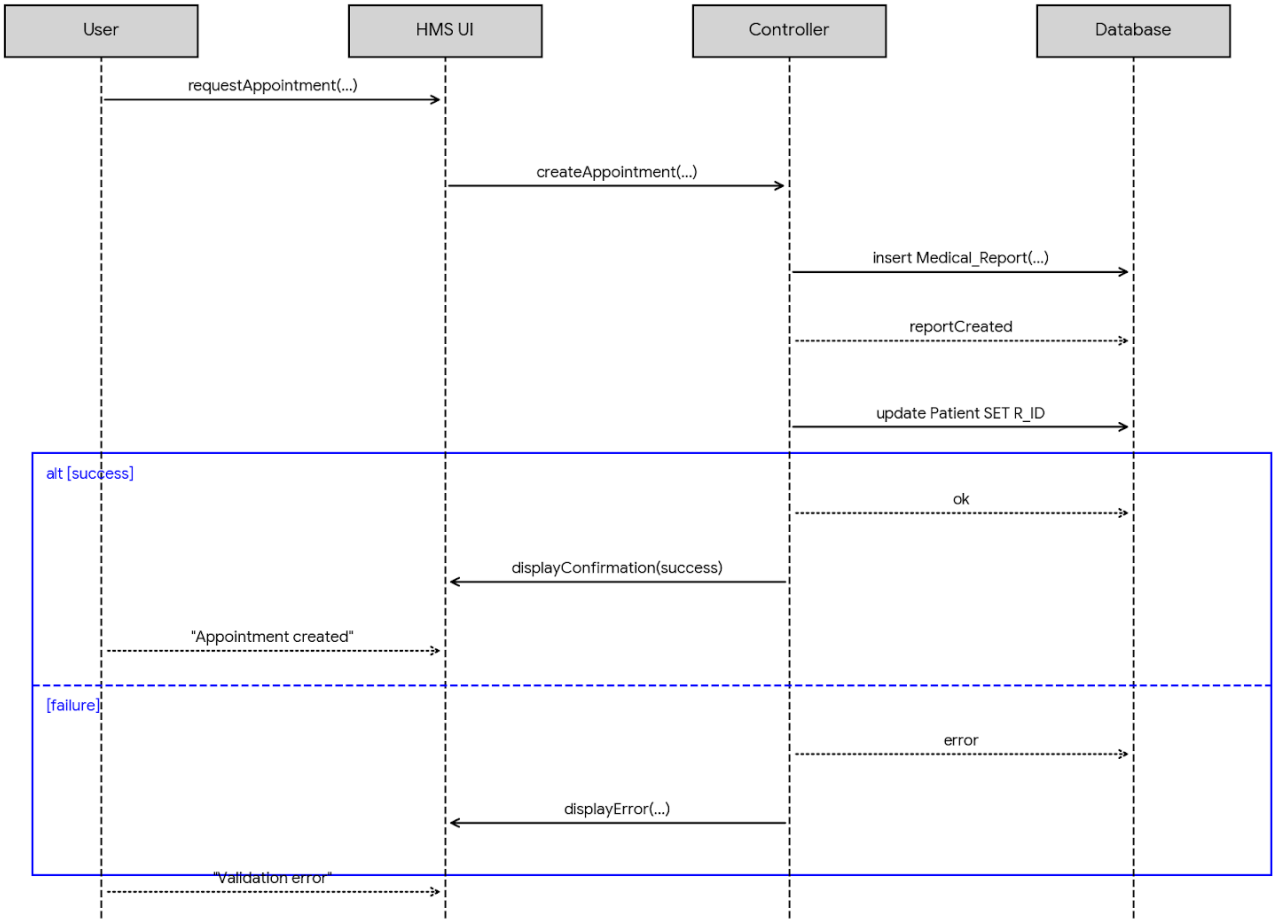
Flow: User requests appointment creation. UI calls controller. Controller creates Medical_Report and updates Patient.R_ID. Database persists. Success or validation error returned.

1.6.1.1 PlantUML

```
\NormalTok{@startuml Book\_Appointment}
\NormalTok{actor User}
\NormalTok{participant "HMS UI" as UI}
\NormalTok{participant "AppointmentController" as Ctrl}
\NormalTok{participant "Medical\_Report" as MR}
\NormalTok{participant "Patient" as P}
\NormalTok{database "Database" as DB}
```

```
\NormalTok{User {-}\textgreater{} UI : requestAppointment(patientID, appointmentDate, description)}
\NormalTok{UI {-}\textgreater{} Ctrl : createAppointment(patientID, appointmentDate, description)}
\NormalTok{Ctrl {-}\textgreater{} DB : insert Medical\_Report(R\_ID, Appointment, Description)}
\NormalTok{DB {-}{-}\textgreater{} Ctrl : reportCreated}
\NormalTok{Ctrl {-}\textgreater{} DB : update Patient SET R\_ID WHERE P\_ID = patientID}
\NormalTok{alt update success}
\NormalTok{ DB {-}{-}\textgreater{} Ctrl : ok}
\NormalTok{ Ctrl {-}\textgreater{} UI : displayConfirmation(success)}
\NormalTok{ UI {-}\textgreater{} User : "Appointment created"}
\NormalTok{else update failure (invalid P\_ID or Room)}
\NormalTok{ DB {-}{-}\textgreater{} Ctrl : error}
\NormalTok{ Ctrl {-}\textgreater{} UI : displayError(validation\_error)}
\NormalTok{ UI {-}\textgreater{} User : "Validation error"}
\NormalTok{end}
\NormalTok{@enduml}
```

Sequence Diagram: Book Appointment



1.6.2 7.2 Upload Lab Test Result

Participants: Lab Technician, Lab UI, LabController, Test_Report, Used_In, Equipment, Database.

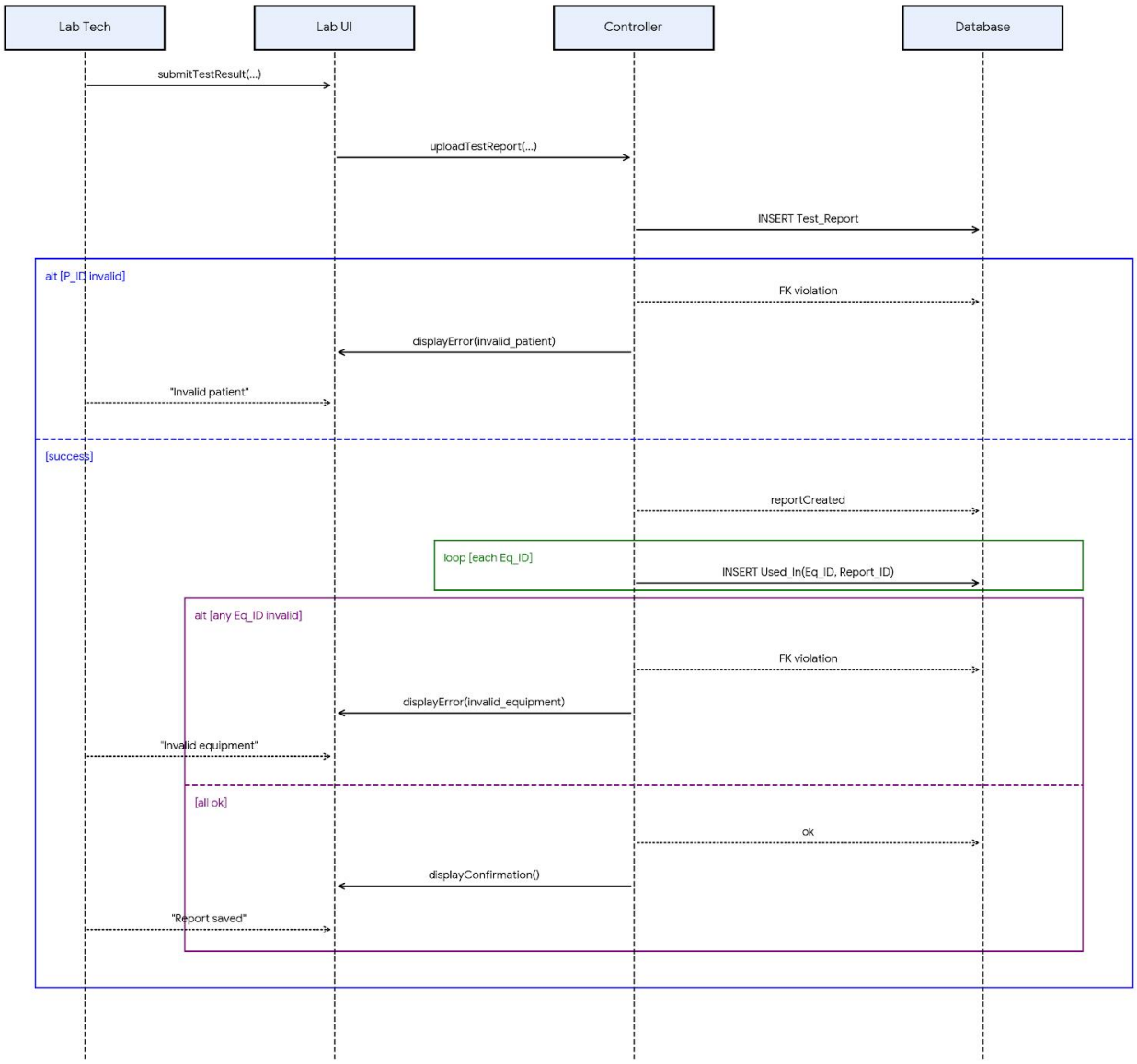
Flow: Technician enters test data and equipment used. Controller creates Test_Report then Used_In rows. Database enforces FKs. Confirmation or error (invalid P_ID/Eq_ID).

1.6.2.1 PlantUML

```
\NormalTok{@startuml Upload\_Lab\_Test\_Result}
\NormalTok{actor "Lab Technician" as LT}
\NormalTok{participant "Lab UI" as UI}
\NormalTok{participant "LabController" as Ctrl}
\NormalTok{participant "Test\_Report" as TR}
\NormalTok{participant "Used\_In" as U}
\NormalTok{database "Database" as DB}

\NormalTok{LT {-}\textgreater{} UI : submitTestResult(patientID, result, appointment, description,
\NormalTok{UI {-}\textgreater{} Ctrl : uploadTestReport(patientID, result, appointment, description,
\NormalTok{Ctrl {-}\textgreater{} DB : insert Test\_Report(Report\_ID, T\_Result, Appointment, Descr
\NormalTok{alt P\_ID invalid}
\NormalTok{ DB {-}{-}\textgreater{} Ctrl : FK violation}
\NormalTok{ Ctrl {-}\textgreater{} UI : displayError(invalid\_patient)}
\NormalTok{ UI {-}\textgreater{} LT : "Invalid patient"}
\NormalTok{else success}
\NormalTok{ DB {-}{-}\textgreater{} Ctrl : reportCreated}
\NormalTok{ loop for each Eq\_ID in equipmentIds}
\NormalTok{  Ctrl {-}\textgreater{} DB : insert Used\_In(Eq\_ID, Report\_ID)}
\NormalTok{  end}
\NormalTok{ alt any Eq\_ID invalid}
\NormalTok{  DB {-}{-}\textgreater{} Ctrl : FK violation}
\NormalTok{  Ctrl {-}\textgreater{} UI : displayError(invalid\_equipment)}
\NormalTok{  UI {-}\textgreater{} LT : "Invalid equipment"}
\NormalTok{ else all ok}
\NormalTok{  DB {-}{-}\textgreater{} Ctrl : ok}
\NormalTok{  Ctrl {-}\textgreater{} UI : displayConfirmation(reportSaved)}
\NormalTok{  UI {-}\textgreater{} LT : "Report saved"}
\NormalTok{  end}
\NormalTok{end}
```

Sequence Diagram: Upload Lab Test Result



```
\NormalTok{@enduml}
```

1.6.3 7.3 Generate Patient Bill

Participants: Administrator, Billing UI, BillingController, Patient_Bill, Service, Patient, Database.

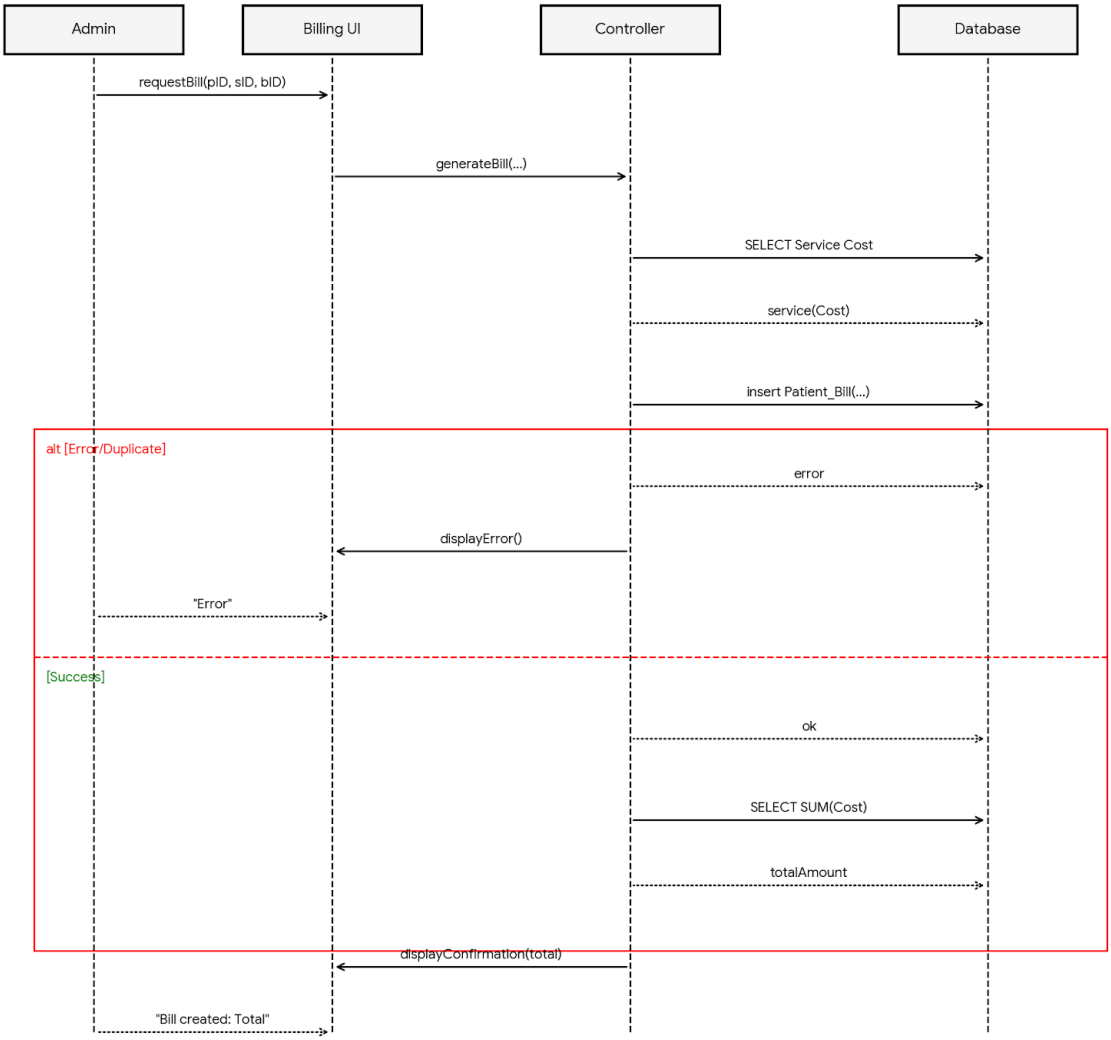
Flow: User selects patient and service. Controller creates Patient_Bill. Database enforces P_ID and Service_ID. Controller may compute total (SUM Service.Cost) for patient and return confirmation.

1.6.3.1 PlantUML

```
\NormalTok{@startuml Generate\_Patient\_Bill}
\NormalTok{actor Administrator}
\NormalTok{participant "Billing UI" as UI}
\NormalTok{participant "BillingController" as Ctrl}
\NormalTok{participant "Patient\_Bill" as PB}
\NormalTok{participant "Service" as S}
\NormalTok{database "Database" as DB}

\NormalTok{Administrator {-}\textgreater{} UI : requestBill(patientID, serviceID, billID)}
\NormalTok{UI {-}\textgreater{} Ctrl : generateBill(billID, patientID, serviceID)}
\NormalTok{Ctrl {-}\textgreater{} DB : SELECT Service WHERE Service\_ID = serviceID}
\NormalTok{DB {-}{-}\textgreater{} Ctrl : service (Cost)}
\NormalTok{Ctrl {-}\textgreater{} DB : insert Patient\_Bill(PB\_ID, P\_ID, Service\_ID)}
\NormalTok{alt duplicate PB\_ID or invalid P\_ID/Service\_ID}
\NormalTok{ DB {-}{-}\textgreater{} Ctrl : error}
\NormalTok{ Ctrl {-}\textgreater{} UI : displayError(invalid\_or\_duplicate)}
\NormalTok{ UI {-}\textgreater{} Administrator : "Error"}
\NormalTok{else success}
\NormalTok{ DB {-}{-}\textgreater{} Ctrl : ok}
\NormalTok{  Ctrl {-}\textgreater{} DB : SELECT SUM(s.Cost) FROM Patient\_Bill pb, Service s WHERE
\NormalTok{ DB {-}{-}\textgreater{} Ctrl : totalAmount}
\NormalTok{ Ctrl {-}\textgreater{} UI : displayConfirmation(billCreated, totalAmount)}
\NormalTok{ UI {-}\textgreater{} Administrator : "Bill created. Total: totalAmount"}
\NormalTok{end}
\NormalTok{@enduml}
```

Sequence Diagram: Generate Patient Bill



1.7 8. Risk and Failure Analysis

Failure mode	Cause	Effect	Mitigation
Data inconsistency	Application inserts Patient_Bill with wrong P_ID or Service_ID; or Test_Report with wrong P_ID.	Wrong patient charged; wrong patient's test history; incorrect reporting.	Enforce foreign keys; UI/controller validate selected P_ID and Service_ID; audit logs for billing and test creation.

Failure mode	Cause	Effect	Mitigation
Lost embryo record	Accidental DELETE Embryo or overwrite of E_ID; no backup.	Loss of chain-of-custody; inability to trace embryo to patient and equipment.	Restrict DELETE via roles; soft-delete or status flag instead of hard delete; regular backups; audit trail for Embryo changes.
Equipment failure (data view)	Equipment record deleted or Eq_ID changed while Used_In or Embryo still reference it.	FK violation on insert/update; or orphan references if FK not enforced.	Enforce FK; do not delete Equipment if referenced; deprecate by Condition or status instead of delete.
Billing mismatch	Duplicate Patient_Bill for same logical service (e.g. same cycle); or Service.Cost changed after bill created.	Patient overcharged or total bill inconsistent with expectations.	Business rule: one bill line per (P_ID, Service_ID, cycle/visit) if required; store snapshot of cost on Patient_Bill if historical accuracy is required; otherwise document that total is always SUM(Service.Cost) at query time.
Concurrent access conflict	Two users update same Patient (e.g. R_ID) or same Donation_Bank (InStock) simultaneously.	Lost update or inconsistent read.	Use transactions with appropriate isolation; locking for critical updates.

1.8 References

Relational schema: SQLite (1) (1).sql. UML 2.x. Case study style: Insulin Pump Control System.